Innovación
en Formación
Profesional

Sistemas de Big Data

Actividad Fase 1:

SmartChef

Curso de Especialización en Inteligencia Artificial y Big Data

MODALIDAD ONLINE

Grupo 9:

· Alejando Fernández Fuentes - Data

· Antoni Sureda Cardona - Platform

· Blas Martos Ortega - Machine Learning

· Hugo Barrera Castillo - Project Management / BI

Índice

1. Elevator Pitch **3**
2. Descripción funcional **4**
Descripción general de la aplicación 4
Flujo general de uso 4
Casos de uso principales 4
Valor para el usuario 5
3. Alcance del Proyecto **6**
IN: Qué incluye el proyecto 6
OUT: Qué no incluye el proyecto 6
4. Arquitectura del sistema **7**
Fuentes de Datos 7
Procesamiento y Almacenamiento 7
Núcleo Analítico y Backend 7
Presentación / Frontend 8
5. Tecnologías Seleccionadas **9**
6. Modelo de datos **10**
Esquema general 10
Relación clave: 10
Conclusión 13
7. DATOS **14**
Fuentes de datos 14
Licencia 14
Volumen estimado 15
Frecuencia de actualización 15
Calidad esperada 15
Riesgos de datos 15
Naturaleza de los datos 16
Conclusión 16
Presencia o ausencia de datos personales (PII) 17
Cumplimiento de GDPR 17
Medidas de mitigación 18
Accesibilidad 18
Conclusión 18
9. Historias de Usuario **19**
Ver predicción de ingredientes 19
Descripción 19
Criterios 19
Consultar dashboard 19
Descripción 19
Criterios 19
Comparar histórico vs predicción 19
Descripción 19

Criterios..... 19
Ver impacto del clima..... 20
Descripción.....20
Criterios..... 20
Consultar reservas futuras..... 20
Descripción.....20
Criterios..... 20
Identificar riesgo de desperdicio.....20
Descripción.....20
Criterios..... 20
Obtener recomendación de compra.....20
Descripción.....20
Criterios..... 20
Filtrar datos por fecha..... 21
Descripción.....21
Criterios..... 21
Visualizar ahorro estimado.....21
Descripción.....21
Criterios..... 21
Acceso al sistema..... 21
Descripción.....21
Criterios..... 21
10. Diagrama de gantt.....22
11. Estrategia de desarrollo.....23
12. Riesgos Técnicos (Matriz de Gestión)..... 24
13. Acceso al repositorio.....25

SmartChef

1. Elevator Pitch

Los restaurantes pierden dinero cada semana por una mala previsión de demanda, generando desperdicio de alimentos y roturas de stock. Proponemos una IA que prediga los kilos necesarios de cada ingrediente perecedero para la próxima semana utilizando histórico de ventas, reservas, clima y festividades locales. El sistema permitirá optimizar compras, reducir desperdicio y mejorar la rentabilidad operativa.

2. Descripción funcional

Descripción general de la aplicación

La aplicación **SmartChef** es una herramienta de apoyo a la toma de decisiones en restaurantes que permite predecir la demanda de ingredientes perecederos a partir de datos históricos, reservas, condiciones meteorológicas y eventos.

Su objetivo principal es ayudar al gerente, jefe de cocina o responsable de compras a optimizar el aprovisionamiento semanal, reduciendo el desperdicio alimentario y evitando roturas de stock.

El sistema presenta la información a través de una interfaz visual sencilla (dashboard), donde el usuario puede consultar predicciones, analizar datos históricos y obtener recomendaciones claras de compra.

Flujo general de uso

El uso de la aplicación sigue un flujo orientado a la toma de decisiones:

1. El usuario accede a la aplicación a través de la interfaz (dashboard).
2. Una vez dentro, visualiza un resumen general con las predicciones de demanda para la próxima semana.
3. El usuario puede consultar el detalle por ingrediente, donde se muestran las cantidades estimadas necesarias.
4. Se presenta una comparación entre datos históricos y predicciones, permitiendo evaluar la evolución del consumo.
5. El sistema muestra indicadores de riesgo, identificando ingredientes con posible exceso (desperdicio) o falta de stock.
6. El usuario revisa las recomendaciones de compra generadas por el sistema.

Con esta información, el gerente o responsable de compras toma decisiones sobre los pedidos a proveedores.

Casos de uso principales

Los principales usos del sistema desde el punto de vista del usuario son los siguientes:

1. **Consulta de predicciones:** El usuario puede visualizar la demanda estimada de ingredientes para la próxima semana, desglosada por producto.
2. **Análisis de histórico vs predicción:** El sistema permite comparar el consumo pasado con las predicciones generadas, facilitando la validación del modelo.
3. **Identificación de riesgo de desperdicio:** El usuario puede detectar qué ingredientes tienen mayor probabilidad de no ser utilizados, permitiendo ajustar las compras.

4. **Recomendación de compras:** La aplicación ofrece una estimación de las cantidades óptimas a comprar, basadas en las predicciones del modelo.
5. **Análisis de factores externos:** El usuario puede entender cómo variables como el clima, las reservas o los eventos influyen en la demanda.

Evaluación del impacto económico: El sistema proporciona una estimación del ahorro potencial derivado de una mejor planificación del inventario.

Valor para el usuario

La aplicación permite transformar datos complejos en información clara y accionable, facilitando la toma de decisiones operativas en el restaurante.

Los principales beneficios són:

- Reducir el desperdicio
- Optimizar los costes de aprovisionamiento
- Mejorar la planificación del negocio

Aumentar la rentabilidad operativa

3. Alcance del Proyecto

IN: Qué incluye el proyecto

- Desarrollo de un modelo predictivo de Machine Learning para estimar la demanda semanal de ingredientes perecederos críticos.
- Creación de un pipeline de datos que combine el histórico de ventas con variables externas como meteorología y festivos.
- Despliegue de una API básica para servir las predicciones del modelo.
- Creación de un dashboard o interfaz visual para la consulta de cantidades recomendadas por parte del gerente.

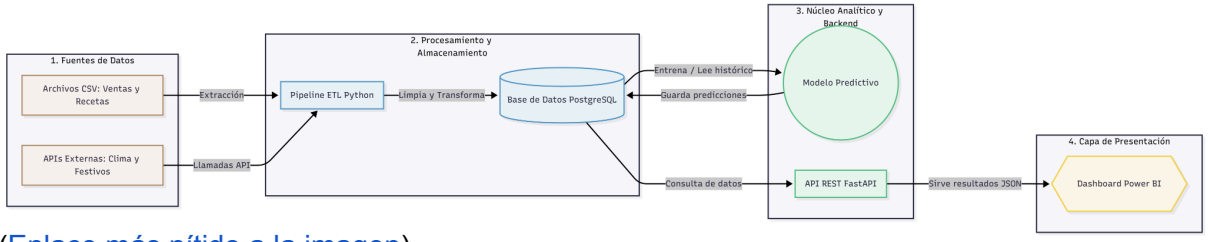
OUT: Qué no incluye el proyecto

- Integración en tiempo real con software de TPV real (se usarán datos estáticos o aislados).
- Desarrollo de un sistema automatizado de compras.
- Predicción de la demanda del 100% del inventario.
- Gestión física del almacén o control de mermas por caducidad en tiempo real.

Funcionalidades Opcionales (NUEVO) Se abordarán únicamente si el cronograma de desarrollo lo permite:

- Simulador de Escenarios "What-if": Ajuste manual de variables externas para observar cambios en la predicción.
- Alertas de Stock de Seguridad: Notificaciones automáticas cuando la demanda prevista supere las existencias críticas.
- Exportación de Pedidos: Generación de formatos listos para enviar a proveedores habituales.

4. Arquitectura del sistema



(Enlace más nítido a la imagen)

La arquitectura de **SmartChef** se ha diseñado bajo un enfoque modular, dividiendo el sistema en cuatro capas. Al no requerir conexión en tiempo real con el TPV del restaurante, el flujo de datos está orientado al procesamiento por lotes garantizando estabilidad y facilidad de mantenimiento.

A continuación, se detalla el funcionamiento de cada componente de la arquitectura:

Fuentes de Datos

El sistema se alimenta de información proveniente de dos orígenes distintos:

- **Archivos CSV (Ventas y Recetas):** Datos estáticos internos del restaurante que proporcionan el histórico de tickets y la matriz de conversión para saber qué ingredientes componen cada plato.
- **APIs Externas (Clima y Festivos):** Servicios web públicos que aportan el contexto necesario para el modelo (temperatura, lluvia y calendario de festividades).

Procesamiento y Almacenamiento

- **Pipeline ETL (Python):** Extracción de los CSV y llamadas a las APIs. Luego, limpia y
  transforma los datos para generar un dataset unificado.
- **Base de Datos (PostgreSQL):** Actúa como la fuente única de verdad. Almacena de forma estructurada tanto los datos históricos ya limpios como las predicciones futuras generadas por el modelo.

Núcleo Analítico y Backend

- **Servicios ML (Modelo Predictivo):** Es el cerebro analítico del sistema. El algoritmo lee el histórico desde la base de datos, se entrena identificando patrones de consumo y estacionalidad, y finalmente guarda las predicciones calculadas de vuelta en PostgreSQL.
- **Backend (API REST FastAPI):** Consulta los datos de las predicciones en PostgreSQL y los sirve hacia el exterior.

Presentación / Frontend

- Dashboard (Power BI): Es la interfaz visual con la que interactúa el usuario final. Se alimenta consumiendo los resultados JSON servidos por la API. Aquí es donde los datos se transforman en valor de negocio mediante gráficas interactivas, alertas de inventario y recomendaciones claras de compra.

5. Tecnologías Seleccionadas

Tecnología	Rol en el Proyecto	Justificación	Alternativa Descartada
Python	Lenguaje Principal	Estándar en IA con librerías robustas para análisis de datos.	R: Curva de aprendizaje más alta para despliegue de APIs.
FastAPI	Framework de API	Alto rendimiento y validación de datos automática para servir predicciones.	Flask: Más lento y requiere más configuración manual.
PostgreSQL	Base de Datos	Ideal para manejar históricos de ventas y escandallos estructurados.	MongoDB: No es óptimo para relaciones complejas entre recetas.
Scikit-learn	Modelado ML	Librería versátil para modelos de regresión y baselines.	TensorFlow: Demasiado complejo para el volumen de datos inicial.
Power BI	Visualización	Herramienta de BI para el dashboard del gerente aprovechando licencias académicas.	Streamlit: Descartado para priorizar una estética de reporte corporativo.

6. Modelo de datos

El sistema se basa en un modelo de datos relacional orientado a series temporales, que permite integrar información de ventas, reservas, clima y recetas para generar predicciones de demanda de ingredientes.

Esquema general

El modelo se compone de las siguientes entidades principales:

- Ventas
- Platos
- Ingredientes
- Recetas
- Reservas
- Clima
- Calendario

Relación clave:

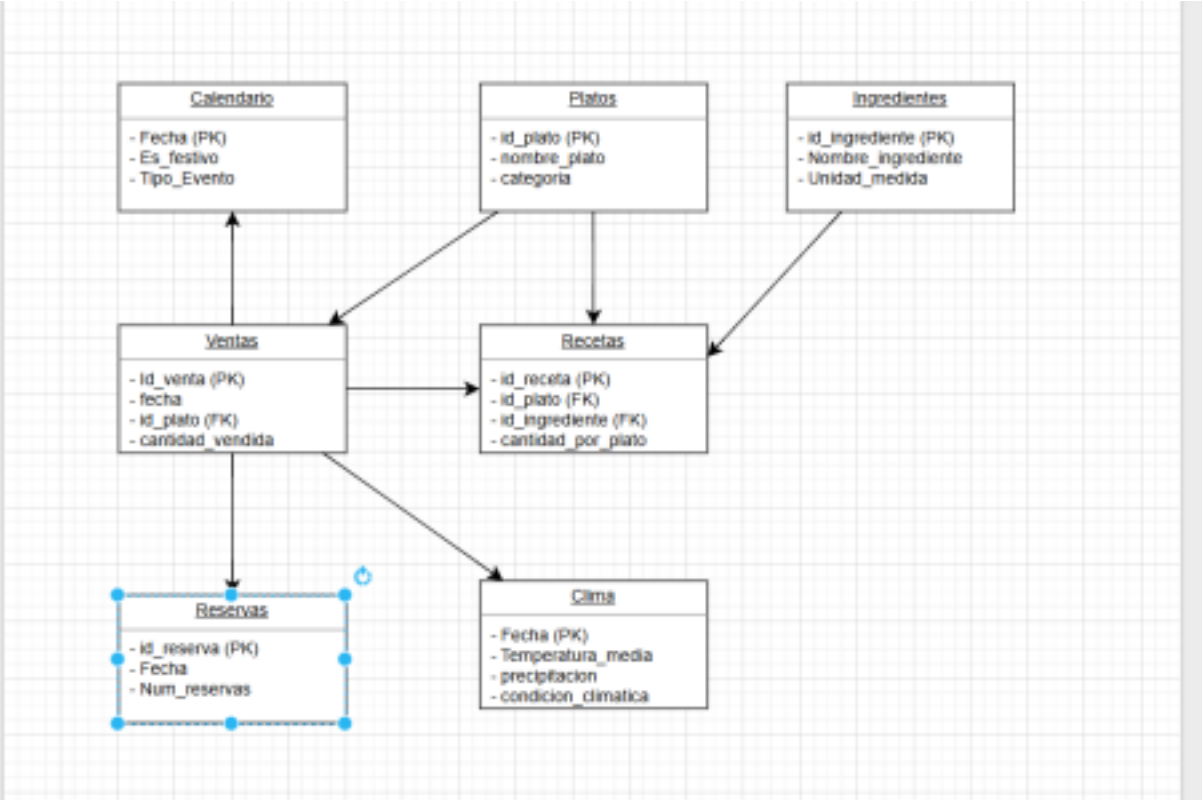
Ventas --> Platos --> Recetas --> Ingredientes

Todo ello enriquecido con datos temporales (clima, reservas, festivos)

Tablas / Colecciones:

- Ventas
    - Id_venta (PK)
    - Fecha
    - Id_plato (FK)
    - Cantidad_vendida
- Platos
    - Id_plato (PK)
    - Nombre_plato
    - Categoria
- Ingredientes
    - id_ingrediente (PK)
    - Nombre_ingrediente
    - Unidad_medida
    - Recetas
    - Id_receta (PK)
    - Id_plato (FK)

    - Id_ingrediente (FK)
    - Cantidad_por_plato
- Reservas
    - Id_reserva (PK)
    - Fecha
    - Num_reservas
- Clima
    - Fecha (PK)
    - Tamperatura_media
    - Precipitacion
    - Condicion_climatica
- Calendario
    - Fecha (PK)
    - Es_festivo (boolean)
    - Tipo_evento
- Relaciones
    - Ventas.id_plato → platos.id_plato (N:1)
    - Recetas.id_plato → platos.id_plato (N:1)
    - Recetas.id_ingrediente → ingredientes.id_ingrediente (N:1)
    - Ventas.fecha → clima.fecha (N:1)
    - Ventas.fecha → clima.fecha (N:1)
    - Ventas.fecha → calendario.fecha (N:1)
    - Ventas.fecha → reservas.fecha (N:1)
- Claves
    - Id_venta, id_plato, id_ingrediente, id_receta, id_reserva
    - Fecha en tablas temporales (clima, calendario)
- Claves foráneas (FK):
    - Id_plato en ventas y recetas
    - Id_ingrediente en recetas



Dataset final para Machine Learning

A partir del modelo anterior, se construirá un dataset agregado con la siguiente estructura:

- Tabla: dataset_modelo
  - Fecha
  - Id_ingrediente
  - Demanda_total
  - Num_reservas
  - Temperatura_media
  - Precipitacion
  - Es_festivo
  - Dia_semana
- Esta tabla será la utilizada para entrenar el modelo predictivo.

Variable	Tipo	Descripción
fecha	datetime	Dia de observación
Id_ingrediente	categórica	Identificador del ingrediente

Demanda_total	numérica	Cantidad total consumida
Num_reservas	numérica	Número de reservas
Temperatura_media	numérica	Temperatura media diaria
precipitacion	numérica	Nivel de lluvia
Es_festivo	binaria	Indica si es festivo
Dia_semana	categórica	Día de la semana

Conclusión

El modelo de datos permite:

- Integrar múltiples fuentes heterogéneas.
- Transformar ventas en demanda real de ingredientes.
- Generar un dataset estructurado para modelos de predicción.

Su diseño es escalable y adaptable a distintos restaurantes, manteniendo coherencia y simplicidad para un MVP.

7. DATOS

El proyecto se basa en la integración de múltiples fuentes de datos heterogéneas con el objetivo de construir un dataset unificado para el entrenamiento de modelos predictivos de demanda de ingredientes.

Fuentes de datos

- **Datos de ventas del restaurante (simulados)**
  - Información de ventas históricas por plato (fecha, cantidad vendida).
  - Generados de forma sintética siguiendo patrones realistas de consumo.
- **Datos de recetas (mockeados)**
  - Relación entre platos e ingredientes con cantidades estimadas.
  - Definidos manualmente para transformar ventas en demanda de ingredientes.
- **Datos de reservas (simulados)**
  - Número de reservas por día.
  - Generados para simular demanda futura.
- **Datos meteorológicos (reales)**
  - Temperatura, precipitaciones, condiciones climáticas.
  - Fuente: API pública (ej. OpenWeather).
- **Calendario de festividades y eventos (reales)**
  - Festivos locales y nacionales.
  - Fuente: datos abiertos gubernamentales o calendarios oficiale

Licencia

- **Datos simulados/mockeados:** Uso libre, generados por el equipo.
- **Datos meteorológicos:** Licencia abierta de uso (según API utilizada, generalmente uso gratuito con limitaciones).
- **Festividades:** Datos públicos de libre uso.

No se utilizarán datos privados ni sujetos a restricciones legales

(PII). Tipo de datos:

- **Tabulares:** ventas, reservas, recetas.
- **Series temporales:** ventas diarias, clima.
- **Categóricos:** tipo de día (laboral/festivo), condiciones climáticas. • **Numéricos:** cantidades vendidas, temperatura, número de reservas.

Volumen estimado

- Entre **1.000 y 10.000 registros** en la fase inicial (dataset sintético).
- Escalable hasta **decenas de miles de registros** si se amplía el horizonte temporal o número de ingredientes.

Frecuencia de actualización

- **Datos históricos**: carga inicial (batch).
- **Datos de clima:** actualización diaria mediante API.
- **Predicciones**: ejecución semanal del modelo.

Calidad esperada

- **Datos simulados:** Alta consistencia estructural, pero menor realismo.
- **Datos reales (clima/festivos):** Alta fiabilidad.
- **Datos de recetas:** Dependientes de la calidad del modelado

Riesgos de datos

- **Datos faltantes**
    - Posibles valores nulos en clima o reservas simuladas.
    - Mitigación: imputación (media, forward fill).
- **Sesgo en datos simulados**
    - Los datos pueden no reflejar comportamientos reales complejos. ○ Mitigación: introducir variabilidad aleatoria y estacionalidad. • Ruido en la demanda

    - Alta variabilidad en consumo diario.
    - Mitigación: agregación semanal y suavizado.
- **Errores en recetas (mock data)**
    - Conversión incorrecta de plato a ingrediente.
    - Mitigación: validación manual de reglas.

Naturaleza de los datos

- **Datos reales:**
    - Clima
    - Festividades
- **Datos simulados:**
    - Ventas
    - Reservas
- **Datos mockeados:**
    - Recetas

Conclusión

El enfoque híbrido (datos reales + simulados + mockeados) permite garantizar la viabilidad del proyecto sin depender de acceso a datos privados, manteniendo un equilibrio entre realismo y control experimental.

8. Ética, legalidad y accesibilidad

El proyecto ha sido diseñado teniendo en cuenta principios de ética en el uso de datos, cumplimiento normativo y accesibilidad, especialmente en lo relativo al tratamiento de datos y al impacto del sistema en la toma de decisiones.

Presencia o ausencia de datos personales (PII)

No se incluirán datos como:

- Nombres de clientes
- Teléfonos
- Emails
- Identificadores personales

Los datos de reservas se tratarán de forma agregada (número de reservas por día), evitando cualquier información individual.

Cumplimiento de GDPR

El proyecto se alinea con el Reglamento General de Protección de Datos (GDPR) mediante:

- **Minimización de datos:** solo se utilizan datos necesarios para el objetivo del modelo.
- **Anonimización:** no se procesan datos identificables.
- **Finalidad específica:** los datos se emplean únicamente para predicción de demanda.
- **Uso de datos públicos o simulados:** evita conflictos legales.

Dado que no se manejan datos personales, el cumplimiento es directo y el riesgo regulatorio es bajo.

Riesgos de sesgo

- Sesgo en datos simulados
    - Los datos generados pueden no reflejar la complejidad real del comportamiento de clientes.

- Sesgo temporal
    - El modelo puede aprender patrones específicos de un periodo concreto (ej. estacionalidad no representativa).
- Sesgo por variables externas
    - Factores como clima o eventos pueden tener un impacto desigual según el contexto.

Medidas de mitigación

Para reducir estos riesgos se aplicarán las siguientes estrategias:

- Introducir variabilidad realista en los datos simulados (ruido, estacionalidad).
- Validación temporal del modelo (train/test cronológico).
- Comparación con baseline para evitar sobreajuste.
- Evaluación por ingrediente para detectar comportamientos anómalos.
- Documentación de limitaciones del modelo (transparencia).

Accesibilidad

En caso de implementación de dashboard (Power BI), se tendrán en cuenta principios básicos de accesibilidad:

- Uso de visualizaciones claras y legibles.
- Evitar colores confusos o poco contrastados.
- Inclusión de etiquetas y descripciones en gráficos.
- Diseño sencillo e intuitivo orientado a usuarios no técnicos.

Conclusión

El proyecto presenta un riesgo ético y legal bajo, al no utilizar datos personales y basarse principalmente en datos simulados y públicos.

Se aplican medidas de mitigación frente a sesgos y se prioriza la transparencia del modelo, garantizando un uso responsable de la inteligencia artificial en el contexto de la restauración.

9. Historias de Usuario

Ver predicción de ingredientes

Descripción

Como gerente quiero ver la predicción semanal de ingredientes para saber cuánto comprar.

Criterios

- Se muestran cantidades por ingrediente
- Se visualizan por semana
- Los datos vienen del modelo predictivo

Consultar dashboard

Descripción

Como gerente quiero acceder a un dashboard visual para entender fácilmente las predicciones.

Criterios

- El dashboard muestra gráficos claros
- Se pueden ver datos históricos y predicción
- Es accesible desde una interfaz simple

Comparar histórico vs predicción

Descripción

Como gerente quiero comparar datos históricos con predicciones para validar si el modelo es fiable.

Criterios

- Se muestran ambos datos juntos
- Se pueden comparar por fechas
- Se visualiza la diferencia

Ver impacto del clima

Descripción

Como gerente quiero ver cómo afecta el clima a la demanda para entender cambios en consumo.

Criterios

- Se muestran datos de clima junto a ventas
- Se reflejan cambios en la predicción
- Se identifican patrones

Consultar reservas futuras

Descripción

Como jefe de cocina quiero ver el número de reservas para anticipar la demanda.

Criterios

- Se muestran reservas por día
- Se integran en la predicción
- Los datos están actualizados

Identificar riesgo de desperdicio

Descripción

Como gerente quiero identificar ingredientes con riesgo de desperdicio para reducir pérdidas.

Criterios

- Se listan ingredientes con exceso previsto
- Se indica nivel de riesgo
- Se actualiza semanalmente

Obtener recomendación de compra

Descripción

Como responsable de compras quiero recibir recomendaciones de cantidades a comprar para optimizar el inventario.

Criterios

- Se muestra cantidad recomendada

- Está basada en predicción
- Se puede consultar por ingrediente

Filtrar datos por fecha

Descripción

Como usuario quiero filtrar datos por fechas para analizar periodos concretos.

Criterios

- Se puede seleccionar rango de fechas
- Se actualizan los gráficos
- Funciona sin errores

Visualizar ahorro estimado

Descripción

Como propietario quiero ver el ahorro estimado para evaluar el impacto económico.

Criterios

- Se muestra ahorro en €
- Se basa en reducción de desperdicio
- Se actualiza con nuevas predicciones

Acceso al sistema

Descripción

Como usuario quiero acceder al sistema para consultar las predicciones.

Criterios

- Se puede acceder a la aplicación
- La interfaz carga correctamente
- Permite navegar por las secciones

10. Diagrama de gantt

El diagrama de Gantt representa la planificación temporal del proyecto, organizando las tareas por fases y asignando responsabilidades a cada miembro del equipo. Se ha estructurado en varias fases (diseño, desarrollo de datos, modelado, integración y validación), permitiendo visualizar la evolución del proyecto de forma progresiva.

Diagrama de gantt

SmartChef

Fase	Responsable	Tarea	# Semana inicio	# Duración (semanas)	Sem 1	Sem 2	Sem 3	Sem 4	Sem 5	Sem 6
Fase 1	Alejandro	Diccionario + ética y legalidad	2	1						
Fase 1	Blas	Alcance + riesgos	1	1						
Fase 1	Blas	Tecnologías + anexo IA	2	1						
Fase 1	Toni	Arquitectura sistema	1	1						
Fase 1	Toni	Estrategia dev + README + repo	2	1						
Fase 2	Hugo	KPIs + diseño dashboard	3	1						
Fase 2	Alejandro	ETL + dataset final	3	1						
Fase 2	Blas	Modelo baseline	3	1						
Fase 2	Toni	Backend API + pipeline	3	1						
Fase 3	Hugo	Validación negocio	4	1						
Fase 3	Alejandro	Feature engineering	4	1						
Fase 3	Blas	Entrenamiento modelos ML	4	1						
Fase 3	Toni	Endpoints predicción	4	1						
Fase 4	Hugo	Diseño dashboard final	5	1						
Fase 4	Alejandro	Ajustes datos	5	1						
Fase 4	Blas	Optimización modelo	5	1						
Fase 4	Toni	Integración + dashboard	5	1						
Fase 5	Hugo	Presentación final	6	1						
Fase 5	Alejandro	Documentación datos	6	1						
Fase 5	Blas	Documentación modelo	6	1						
Fase 5	Toni	Despliegue + revisión repo	6	1						

11. Estrategia de desarrollo

Para garantizar la calidad del código, evitar conflictos entre los miembros del equipo y mantener el repositorio organizado, hemos definido una estrategia de desarrollo basada en un flujo de trabajo estructurado y revisiones continuas.

Adoptaremos un modelo simplificado basado en GitFlow, utilizando el siguiente esquema de ramas:

- **Main:** Contiene únicamente el código en producción, estable, funcional y revisado. No se pueden hacer commits directos a esta rama.
- **Dev:** Actuará como entorno de pruebas conjunto. Aquí es donde unificamos el trabajo de todos antes de pasarlo a producción.
- **Ramas individuales (feature/):** Cada vez que un miembro del equipo empiece un Issue (por ejemplo, crear la base de datos o hacer el modelo), creará una rama nueva desde dev con una nomenclatura clara (ej. **feature/**limpieza-datos o **feature/**api-fastapi).

El proceso para juntar el código será el siguiente:

1. El desarrollador termina su tarea en su rama feature/ asegurándose de realizar commits claros y descriptivos.
2. Se abre un Pull Request hacia la rama dev.
3. Se requiere la revisión de al menos otro miembro del equipo antes de aprobar el merge. 4. Una vez que la rama dev acumula las funcionalidades necesarias para una entrega y se comprueba que todo funciona en conjunto, se realiza un pull request final hacia la rama main.

La validación de funcionalidad se realizará cruzando el código con el backlog inicial. Antes de aprobar cualquier pull request, el revisor deberá comprobar que el código cumple estrictamente con los "criterios de aceptación" definidos en la historia de usuario correspondiente. Si la tarea era "Crear endpoint de predicción", se validará que la API efectivamente devuelve la predicción solicitada sin errores.

Al tratarse de un MVP, no implementaremos un sistema complejo de pruebas automatizadas (TDD continuo), pero sí aplicaremos un testing funcional básico dividido por capas:

- **Testing de Datos (ETL):** Verificación mediante *scripts* de aserciones en Python para comprobar que el dataset final no contiene valores nulos inesperados, que los cruces (JOINs) no duplican filas y que los tipos de datos son correctos.
- **Testing del Modelo (ML):** Comprobación de que el modelo es capaz de ejecutarse sobre datos nuevos sin fallar (pruebas de inferencia) y que sus resultados superan las métricas del modelo baseline.
- **Testing de la API (Backend):** Uso de herramientas como Swagger UI o Postman para lanzar peticiones manuales a los endpoints y verificar que devuelven un código de estado “200 OK” y un JSON con la estructura correcta.

12. Riesgos Técnicos (Matriz de Gestión)

Riesgo Técnico	Probabilidad	Impacto	Plan de Mitigación (Plan B)
Falta de datos reales suficientes	Alta	Crítico	Generar dataset sintético realista basado en estacionalidad.
Presencia de datos personales (PII)	Media	Alto	Anonimización completa o exclusión de información personal.
El modelo no supera al Baseline	Media	Medio	Probar nuevos enfoques de ingeniería de características.
Pipeline demasiado complejo	Baja	Alto	Implementar primero una versión offline con exportación a CSV.
Variabilidad extrema de demanda	Alta	Medio	Trabajar con agregación semanal e incluir intervalos de confianza.

13. Acceso al repositorio

Para facilitar la revisión y el acceso al código fuente del proyecto por parte del profesor, el repositorio de GitHub ha sido configurado como público. Puede acceder a él a través del siguiente enlace: https://github.com/ToniSureda/smartchef